Detección paralela de sobrecargas progresivas

Malak Sanchez
Monitorias Sistemas Operativos

15 de octubre de 2026

Contexto

Una central eléctrica recibe un archivo con millones de lecturas de consumo tomadas en intervalos regulares. Una sobrecarga progresiva ocurre cuando tres lecturas consecutivas crecen con el mismo incremento y su consumo acumulado alcanza un umbral de seguridad. Se requiere localizar una de estas ventanas utilizando varios procesos, sin duplicar el arreglo en la memoria privada de cada hijo.

Los procesos trabajadores consultarán una única copia de las lecturas en memoria compartida. Una señal dará la orden de inicio y una tubería transportará todos los hallazgos al coordinador.

Descripción

Sean x_i, x_{i+1} y x_{i+2} tres lecturas consecutivas. La ventana que comienza en la posición i es una **sobrecarga progresiva válida** si cumple simultáneamente:

$$x_i < x_{i+1} < x_{i+2}, \qquad x_{i+1} - x_i = x_{i+2} - x_{i+1}, \qquad x_i + x_{i+1} + x_{i+2} \geq U.$$

Puede haber cero, una o varias ventanas válidas. El programa debe conservarlas todas e indicar, para cada una, el hijo que la encontró, su posición inicial y las tres lecturas que la forman.

Entrada

Ejecute el programa con la siguiente forma:

```
./detector_sobrecargas <archivo> <P>
```

La primera línea del archivo contiene N y el umbral U. A continuación aparecen exactamente N enteros separados por espacios o saltos de línea.

```
20 120
5 30 50 70 10 40 45 60 75 20
100 130 160 45 25 80 100 120 121 60
```

Se garantiza $3 \leq N \leq 100000000$, $1 \leq P \leq \min(16, N-2)$, $0 \leq x_i \leq 10^9$ y $0 \leq U \leq 3 \times 10^9$. El programa debe rechazar argumentos inválidos, datos faltantes, valores fuera de rango y contenido adicional.

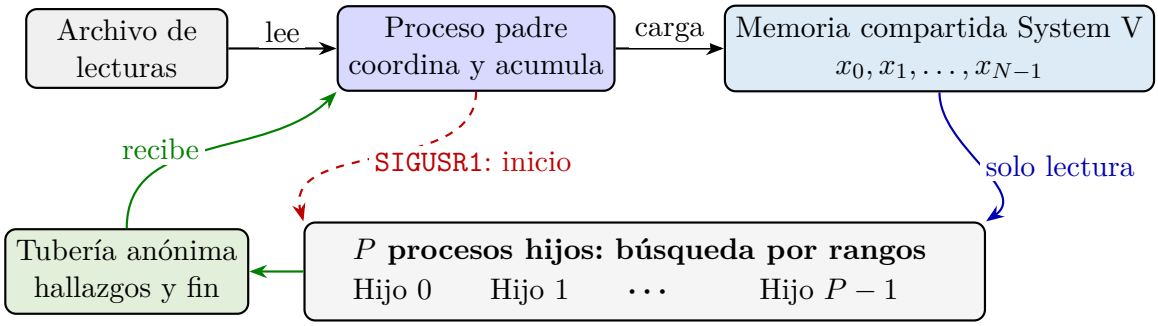
Arquitectura requerida

El padre es el único proceso que abre y lee el archivo. Debe reservar un segmento de memoria compartida System V y cargar allí las N lecturas antes de crear exactamente P hijos. La memoria permanecerá en modo de solo lectura para los trabajadores durante la búsqueda.

El espacio de candidatos contiene $N-2$ índices de inicio. El hijo i, numerado desde cero, examina el intervalo semiabierto

$$\left[\left\lfloor \frac{i(N-2)}{P} \right\rfloor, \left\lfloor \frac{(i+1)(N-2)}{P} \right\rfloor\right).$$

Esta partición debe cubrir todos los índices de inicio exactamente una vez. Dos hijos pueden leer una misma medición en los bordes, pero no pueden evaluar el mismo índice de inicio.



Algoritmo esperado

1. El padre valida la entrada, crea la tubería y la memoria compartida, y almacena allí las N lecturas.
2. El padre crea P hijos y luego envía `SIGUSR1` a cada uno para iniciar la búsqueda. Cada hijo calcula los límites de su intervalo.
3. Para cada índice j, el hijo evalúa la condición usando x_j, x_{j+1} y x_{j+2}. Por cada coincidencia envía un reporte y continúa recorriendo su rango.
4. Al terminar, cada hijo envía un reporte de fin. El padre lee la tubería hasta recibir los P reportes de fin y almacena cada hallazgo en un vector dinámico.
5. El padre ordena los hallazgos por posición, imprime el total y el detalle de cada uno, espera a los hijos y libera los recursos.

Requerimientos

- Implementar la solución en C para Linux usando `fork()`, memoria compartida System V, una tubería anónima y señales POSIX.
- Solo el padre lee el archivo; cada hijo examina únicamente el intervalo que le corresponde.
- Usar `SIGUSR1` como orden de inicio después de crear todos los procesos trabajadores.
- Enviar un reporte por cada hallazgo y un reporte de fin por cada hijo. La estructura debe incluir tipo, trabajador, índice y lecturas.
- Guardar los hallazgos en un vector dinámico y ordenarlos de menor a mayor posición antes de imprimirlos.
- Cerrar los extremos de tubería, esperar a todos los hijos y eliminar la memoria compartida.
- No usar hilos, otros mecanismos IPC, archivos auxiliares, espera activa ni `sleep()`.
- Compilar sin advertencias con `-std=c11 -Wall -Wextra -Wpedantic`.

Ejemplo de salida

Para $P = 4$ y el archivo mostrado, la salida esperada es:

```
MONITOR DE SOBRECARGAS
COINCIDENCIAS: 4
HIJO: 0  POSICION: 1   LECTURAS: 30 50 70
HIJO: 1  POSICION: 6   LECTURAS: 45 60 75
HIJO: 2  POSICION: 10  LECTURAS: 100 130 160
HIJO: 3  POSICION: 15  LECTURAS: 80 100 120
```

Si no existe una ventana válida, la salida debe incluir:

```
COINCIDENCIAS: 0
```

Los reportes pueden llegar en cualquier orden, pero la salida final debe estar ordenada por posición. Para el archivo de ejemplo, las posiciones pertenecen respectivamente a los rangos de los hijos 0, 1, 2 y 3.